

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

И.Д. Свеженин

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

Функциональное программирование

по курсу: Кроссплатформенное программирование

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2025

СОДЕРЖАНИЕ

1. Задание	2
2. Ход работы.....	3
3. Результат работы программы.....	4
4. Вывод.....	5
5. Листинг с кодом программы.....	6

1. Задание

Для выполнения первой лабораторной работы программа должна вычислять значение заданной функции путем разложения в ряд Маклорена с заданной точностью и с использованием стандартной функции класса Math. Аргумент функции и точность должны задаваться пользователем. Ввод и вывод информации можно осуществлять через командную строку. При разработке следует придерживаться принципов функционального программирования. На рисунке 1 представлен задание для первого варианта.

$$1. e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \frac{x^5}{5!} + \dots + \frac{x^n}{n!} + \dots$$

Рисунок 1 – Задание вариант номер 1

2. Ход работы

В ходе выполнения лабораторной работы была разработана программа на языке Kotlin, позволяющая вычислять значение функции e^x в степени x с использованием разложения в ряд Маклорена. Пользователь вводит значение аргумента и требуемую точность, после чего программа рассчитывает сумму ряда до тех пор, пока очередной член не станет меньше заданного порога. Для решения задачи были реализованы отдельные функции: рекурсивная функция факториала, функция вычисления отдельного члена ряда и функция суммирования ряда с контролем точности. Такой подход позволяет структурировать программу и сделать её более читаемой и удобной для сопровождения.

При написании программы применялись принципы функционального программирования. Основные вычисления оформлены в виде чистых функций, которые зависят только от входных параметров и не изменяют внешнее состояние. Использование рекурсии в функции факториала демонстрирует характерный приём функционального стиля, а работа с неизменяемыми переменными повышает предсказуемость программы. Кроме того, вычислительная логика была отделена от ввода и вывода данных, что упрощает тестирование и делает программу ближе к функциональной парадигме.

3. Результат работы программы

В качестве результата работы программы представлен вывод значений в терминал: Значение e в степени x с точностью ϵ : result. На рисунках 1-3 показан пример работы программы с различными значениями.

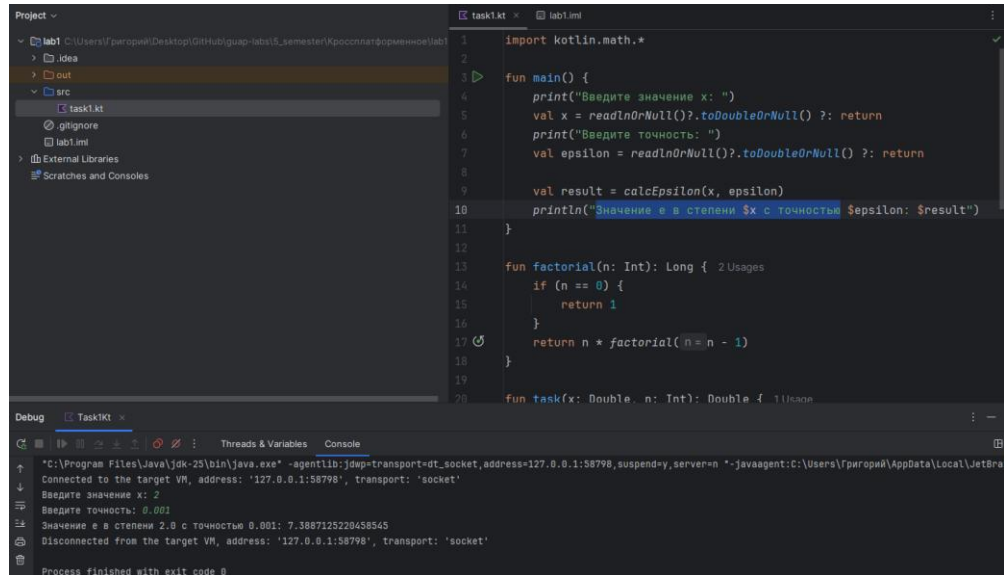


Рисунок 2 – Результат при значениях 1; 0.001.

```
Введите значение x: 4
Введите точность: 0.1
Значение e в степени 4.0 с точностью 0.1: 54.54818021484688
```

Рисунок 3 – Результат при значениях 4; 0.1.

```
Введите значение x: 21
Введите точность: 0.00001
Значение e в степени 21.0 с точностью 1.0E-5: 6.211284016069584E8
```

Рисунок 4 – Результат при значениях 21; 0.00001.

4. Вывод

В результате выполнения лабораторной работы была создана программа на языке Kotlin, которая вычисляет значение функции e^x в степени x методом разложения в ряд Маклорена с заданной точностью. Реализация показала, что использование функционального подхода делает программу более структурированной и наглядной.

Применение чистых функций, рекурсии и работы с неизменяемыми данными позволило приблизить решение к принципам функционального программирования. Таким образом, поставленная задача была успешно решена, а работа продемонстрировала возможности применения функционального стиля при разработке программ на Kotlin.

5. Листинг с кодом программы

В данном разделе представлен исходный код моей части программы:

```
//Kotlin

import kotlin.math.*

fun main() {
    print("Введите значение x: ")
    val x = readlnOrNull()?.toDoubleOrNull() ?: return
    print("Введите точность: ")
    val epsilon = readlnOrNull()?.toDoubleOrNull() ?: return

    val result = calcEpsilon(x, epsilon)
    println("Значение e в степени $x с точностью $epsilon: $result")
}

fun factorial(n: Int): Long {
    if (n == 0) {
        return 1
    }
    return n * factorial(n - 1)
}

fun task(x: Double, n: Int): Double {
    return x.pow(n) / factorial(n).toDouble()
}

fun calcEpsilon(x: Double, epsilon: Double): Double {
    var result = 1.0
    var n = 1
```

```
while (true) {  
    val local = task(x, n)  
    if (local < epsilon) {  
        break  
    }  
    result += local  
    n++  
}  
  
return result  
}
```